Incremental Algorithms: An Exploration into SVRG & SDCA

Boray Kasap, Lingyi Liu, Luca Pana

EPFL, January 2026

1 Theoretical Foundation

We consider the standard regularized empirical risk minimization problem:

$$\min P(w), \qquad P(w) := \frac{1}{n}\sum_{i=1}^{n}\psi_i(w) \; + \; \frac{\lambda}{2}\|w\|^2 \tag{1}$$

Here, $\psi_i(w)$ denotes the loss incurred on the i-th data sample, and the term $\frac{\lambda}{2}\|w\|^2$ represents the ℓ_2-regularization penalty, which is commonly utilized to control model complexity and prevent overfitting.

1.1 Gradient Descent

One general method to solve the minimization problem is Gradient Descent (GD). GD iteratively updates the parameter vector in the direction that most effectively decreases the objective function. To perform this update, we compute the full gradient of the objective function:

$$\nabla P(w) = \frac{1}{n}\sum_{i=1}^{n}\nabla\psi_i(w). \tag{2}$$

The update rule given the gradient is given by:

$$w_{t+1} = w_t - \eta\nabla P(w_t), \tag{3}$$

where $\eta > 0$ denotes the learning rate.

As shown in Nesterov (2013), when the objective function $\nabla P(w)$ is strongly convex and has a Lipschitz continuous gradient, choosing a sufficiently small step size ensures linear convergence. More precisely, GD satisfies:

$$P(w_t) - P(w^\star) = O(r^t), \qquad r < 1, \tag{4}$$

However, problems related to computational complexity arise. Each iteration in GD involves computing the derivatives for all n data samples at each iteration. Hence, GD becomes too expensive for modern large-scale machine learning tasks. This motivates the use of stochastic optimization methods, which reduces per-iteration computational cost.

1.2 Stochastic Gradient Descent

Stochastic Gradient Descent (SGD) replaces the full gradient with an unbiased single-sample estimate, originally studied in Robbins and Monro (1951) and later developed extensively in modern large-scale learning (Bottou et al. 2018). It hence reaching only $1/n$ per-iteration complexity of that of full Gradient Descent. In iteration t, we draw an index i_t uniformly at random from $\{1, \ldots, n\}$ and compute the stochastic gradient:

$$g_t = \nabla\psi_{i_t}(w_t). \tag{5}$$

and the update rule becomes:

$$w_{t+1} = w_t - \eta_t g_t, \tag{6}$$

However, the randomness in the gradient estimate introduces variance, causing the iterates to fluctuate around the optimum. As a consequence, SGD typically converges much more slowly than GD. With a decaying learning rate such as $\eta_t = O(1/t)$, SGD achieves the classical sublinear convergence rate

$$P(w_t) - P(w^\star) = O(1/t), \tag{7}$$

which is significantly slower than the linear convergence rate obtained by GD under strong convexity. These limitations motivate the development of improved stochastic methods that aim to reduce the variance of the gradient estimator while allowing the use of constant or larger learning rates.

1.3 Stochastic Variance Reduced Gradient

Stochastic Variance Reduced Gradient (SVRG) periodically computes a full gradient at a reference point, called the snapshot, and uses this information to construct a corrected gradient with much lower variance.

At the beginning of each epoch, we select a reference point $\tilde{w}$ and compute a full gradient

$$\tilde{\mu} = \nabla P(\tilde{w}) = \frac{1}{n}\sum_{i=1}^{n} \nabla \psi_i(\tilde{w}), \tag{8}$$

During the epoch, SVRG performs m inner-loop updates. At each inner iteration, a random index i_t is selected uniformly from $\{1, \ldots, n\}$, and the algorithm updates the parameter vector using the variance-reduced gradient estimator:

$$w_t = w_{t-1} - \eta \left(\nabla \psi_{i_t}(w_{t-1}) - \nabla \psi_{i_t}(\tilde{w}) + \tilde{\mu} \right). \tag{9}$$

In SVRG, evaluating the full gradient once per epoch amortizes the cost, resulting in an overall runtime of $O((n+\kappa)\log(1/\varepsilon))$, derived from Theorem 1 in Johnson and Zhang 2013. This matches the optimal complexity of first-order methods for strongly convex objectives while maintaining only $O(1)$ per-iteration work.

Since the variance of the gradient estimator decreases within each epoch, SVRG can use a constant step size, leading to significantly faster convergence. Under strong convexity and smoothness, SVRG achieves an overall linear convergence rate of the form

$$P(w^{(s)}) - P(w^\star) = O(\alpha^{\,s}), \qquad \alpha < 1, \tag{10}$$

1.4 Stochastic Dual Coordinate Ascent

Stochastic Dual Coordinate Ascent (SDCA) solves empirical risk minimization problems by operating on the dual information of the objective. The primal problem in (1) has the dual object with form

$$D(\alpha) = -\frac{1}{n}\sum_{i=1}^{n} \psi_i^*(-\alpha_i) - \frac{1}{2\lambda}\left\| \frac{1}{n}\sum_{i=1}^{n} \alpha_i x_i \right\|^2, \tag{11}$$

where ψ_i^* denotes the convex conjugate of ψ_i. A standard stopping criterion is the duality gap $P(w_k) - D(\alpha_k)$, which yields an upper bound on the primal suboptimality(Shalev-Shwartz and Zhang 2013).

At each iteration, SDCA selects an index i uniformly at random and solves a one-dimensional optimization problem to determine the optimal dual update:

$$\Delta\alpha_i \leftarrow \arg\max_{\Delta} \left\{ -\psi_i^*\big(-(\alpha_i^{(t-1)} + \Delta)\big) - \frac{\lambda n}{2}\left\| w^{(t-1)} + (\lambda n)^{-1}\Delta x_i \right\|^2 \right\}. \tag{12}$$

The dual and primal variables are then updated as

$$\alpha_i^{(t)} = \alpha_i^{(t-1)} + \Delta\alpha_i e_i, \qquad w^{(t)} = w^{(t-1)} + (\lambda n)^{-1}\Delta\alpha_i x_i. \tag{13}$$

SDCA implicitly achieves a similar effect. At the optimal solution $(w^\star, \alpha^\star)$, dual feasibility implies that $\nabla\psi_i(w^\star) + \alpha_i^\star = 0$ for every i. Thus, as $(w^{(t)}, \alpha^{(t)})$ approaches $(w^\star, \alpha^\star)$, the dual update $\Delta\alpha_i^{(t)}$ (and consequently the primal update $\Delta w^{(t)} = \frac{\Delta\alpha_i^{(t)}}{\lambda n} x_i$) naturally shrinks to zero without requiring diminishing step sizes. This property implies variance reduction in an implicit way.

Additionally, convex duality implies that at the optimum, all dual coordinate gradients are zero, ensuring that SDCA's updates vanish automatically as the iterates converge, conceptually solving the limitation introduced by SGD, where the updates have expectancy 0 but might have bigger values due to variance. Under smoothness assumptions, SDCA achieves also linear convergence.

2 Methods

We perform optimization on the empirical risk minimization problem (1) using the squared loss

$$\ell_i(w, b) = (y_i - x_i^\top w - b)^2,$$

together with ℓ_2 regularization on both weights and bias. To handle the bias term uniformly across methods, we augment the feature matrix with a column of ones and extend the weight vector accordingly. This ensures all three algorithms optimize the same objective for accurate comparison.

For SGD and SVRG, optimization terminates when consecutive objective values satisfy $|P(w_{t+1}) - P(w_t)| < \varepsilon$. For SDCA, we terminate when the duality gap satisfies $P(w_t) - D(\alpha_t) < \varepsilon$.

3 Experiments

We evaluate the proposed optimization methods on the Bank Marketing dataset from the UCI Machine Learning Repository UCI Machine Learning Repository (2012). The dataset consists of client-related and campaign-related attributes collected from a Portuguese banking institution, with the goal of predicting whether a client subscribes to a term deposit.

The dataset contains both numerical and categorical features, and the target variable is binary, indicating subscription (`yes`) or non-subscription (`no`). Following standard practice, we encode the target variable as $y \in \{0, 1\}$. The data set contains 45211 instances and 16 features

3.1 Variance Reduction Method of SVRG

We apply a constant learning rate for SVRG, with a decaying step size of the form $\eta_t = 1/(1 + \alpha * t)$ for SGD, for α a dampening hyperparamter, in accordance with their theoretical convergence guarantees. The number of inner-loop iterations is set to $m = 20$.

To demonstrate the variance reduction effect of SVRG relative to SGD, we first perform a hyperparameter search for both algorithms. The hyperparameters considered are the learning rate and the regularization parameter λ. Details of the hyperparameter search procedure and the explored parameter ranges are reported in Appendix. The best performance (in terms of accuracy) for SVRG is achieved with a learning rate of $\eta = 10^{-3}$ and a regularization parameter of $\lambda = 10^{-4}$. For SGD, the best-performing configuration uses $\eta = 10^{-2}$ and $\lambda = 10^{-1}$. More results can be seen in Tables 2 and 3.

The test losses convergence are shown in Figure 2 . We first observe that SVRG requires significantly fewer iterations to converge compared to SGD. Moreover, SVRG exhibits a smoother and more monotonic convergence behavior as an effect of variance reduction, whereas SGD is not only slower but also shows some pronounced fluctuations. In particular, the training loss for SGD does not decrease monotonically and occasionally exhibits sudden jumps.

3.2 Effects of dual optimization of SDCA

Unlike SVRG and SGD, SDCA operates in the dual space and does not require a learning rate hyperparameter. Each coordinate update has a closed-form solution, making the regularization parameter λ the only hyperparameter to tune. We perform a hyperparameter search over $\lambda \in \{10^{-6}, 10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}\}$.

We also investigate the effect of initialization on SDCA's convergence. While the standard approach initializes all dual variables $\alpha_i = 0$, we additionally test warm starting by first running a brief adaptation of SGD initialization and setting the dual variables from the resulting primal solution, as suggested in the original paper (Shalev-Shwartz and Zhang 2013) . Specifically, for the warm start (Modified-SGD), after processing sample i with current weights w_t, we set

$$\alpha_i = \frac{y_i - w_t^\top x_i - b_t}{\frac{\|x_i\|^2+1}{\lambda t} + \frac{1}{2}}$$

for the squared loss, where t is the number of samples seen so far.

Table 4 shows that both initialization strategies exhibit similar patterns across regularization values, with peak performance around $\lambda = 10^{-4}$ to 10^{-5}, with the Modified-SGD warm up showing on par or better accuracy, while also having a slightly better convergence rate, as expected from theoretical

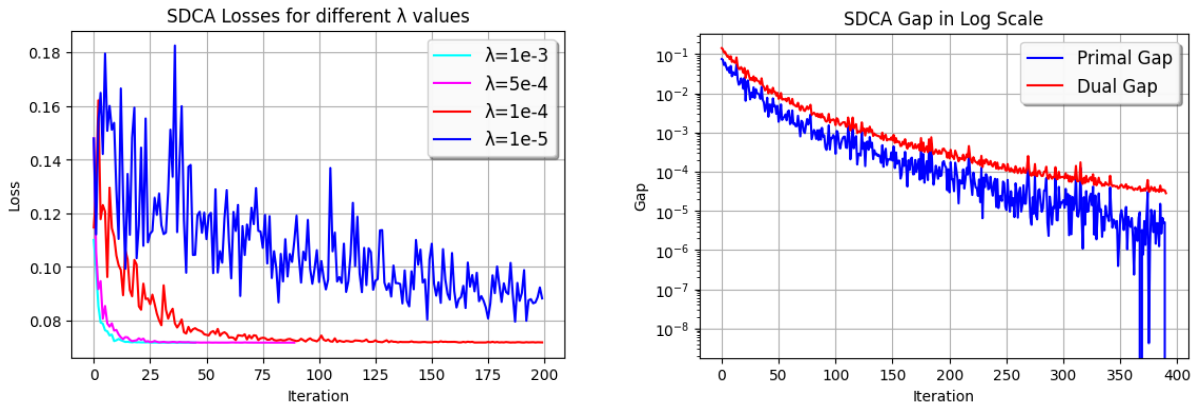

Figure 1: a) The left image shows the convergence of the losses for different lambda hyperparamter values for SDCA b) The right image shows the curves of the duality gap and the primal (empirical) gap $P(w) - P(w_{final})$ as a function of iteration for the SDCA algorithm

results. Figure 1 shows the impact of the regularization parameter. We observe that for lower values the graph fluctuates more in the beginning of training, while it is smooth for relatively high values. We state that this is emphasized by the second term of the dual formulation of the loss, which uses the inverse of lambda to scale the norm, hence smaller values display initial increased step sizes. The theoretical upper bound of the duality gap over the primal gap can be empirically seen in Figure 1, confirming that minimizing the duality gap decreases also the primal gap, providing a principled stopping criterion.

3.3 Comparison

We compare the convergence behavior and computational efficiency of all four methods using their best-performing hyperparameter configurations. The results reveal distinct trade-offs between computational cost and convergence speed.

GD achieves the smoothest convergence but requires computing the full gradient at each iteration, making it impractical for large-scale problems. SGD, while computationally efficient per iteration, exhibits significant oscillations and converges slowly due to high gradient variance. In contrast, both SVRG and SDCA successfully bridge the gap between computational efficiency and convergence speed, as it can be seen from the convergence in Figure 2.

SVRG seems to converge more smoothly than SDCA by the results in Figures 1 and 4 . SDCA updates one dual variable at a time (coordinate descent), which means each iteration makes a localized change. Unlike gradient-based methods that blend information from different samples, SDCA's updates can cause sudden shifts in the primal objective when individual dual coordinates are corrected, especially early in training when dual variables are far from optimal and for low regularizer values Figure 1

Examining the final test performance, SDCA achieves the best accuracy (0.8980) with a test loss of 0.0732, closely followed by SVRG (0.8958 accuracy, 0.0737 loss). SGD reaches in average lower accuracy and higher test loss. The variance-reduced methods require approximately 3-4× fewer iterations than SGD to converge, confirming the theoretical prediction that variance reduction enables faster convergence rates while maintaining per-iteration costs comparable to SGD.

4 Conclusion

We examine incremental learning algorithms and compare the performance of SVRG and SDCA with traditional SGD and GD, focusing on convergence speed. SVRG effectively reduces the variance in the gradient estimates, allowing faster convergence. On the other hand, SDCA optimizes the dual problem, providing both stability and efficiency through an implicit variance regularization. Memorywise, SDCA needs additional $O(n)$ memory for the dual coordinates, while SVRG needs only $O(d)$, where d is the size of the model, for the gradient snapshot. In practice $d << n$, hence SVRG is more memory efficient. Overall, our experiments show that SVRG and SDCA have similar convergence rates and they both outperform SGD in convergence and accuracy and GD in efficiency, and with better or similar accuracy.

Appendix: Extended Algorithm Comparison

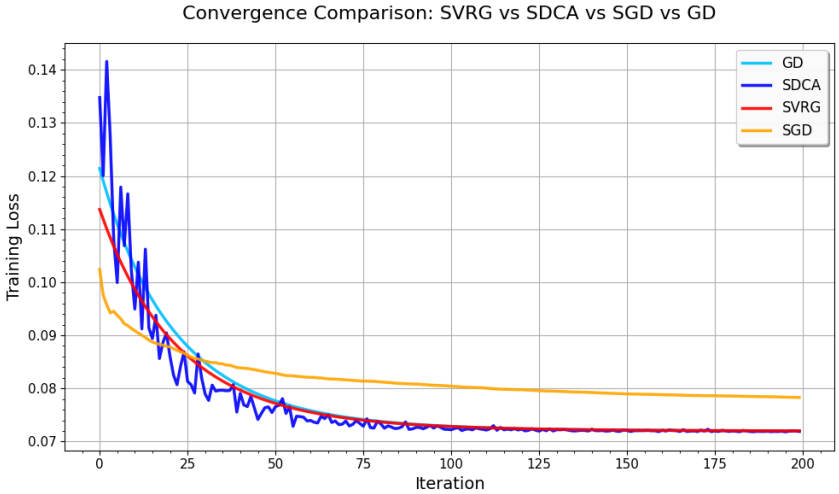

Figure 2: The curves for all training losses: GD, SDCA, SVRG and SGD. Each iteration corresponds to $O(n)$ gradient computations.

Hyperparameters were selected using a grid search over the learning rate η and the regularization parameter λ. The grid search ranges used were:

$$\eta \in \{1 \times 10^{-6}, 5 \times 10^{-5}, 1 \times 10^{-5}, 5 \times 10^{-4}, 1 \times 10^{-4}, 5 \times 10^{-3}, 1 \times 10^{-3}, 1 \times 10^{-2}, 1 \times 10^{-1}\},$$

$$\lambda \in \{1 \times 10^{-6}, 1 \times 10^{-5}, 1 \times 10^{-4}, 1 \times 10^{-3}, 1 \times 10^{-2}, 1 \times 10^{-1}\}.$$

For each method, configurations were evaluated using the validation loss, and the best-performing parameters were retained for subsequent experiments. For readability, only selected results are presented here; full results are available upon request.

We compare also the primal gaps of the different algorithms. From Figure 3 we notice the suboptimal convergence of SGD and the better convergence of the the variance reduction and dual ascent algorithms, as expected from theoretical grounds. Note that as we optimize the dual objective for SDCA, the dual gap is smoother while the primal gap can fluctuate as we can see in Figure 3, but also noticeable in Figure 1.

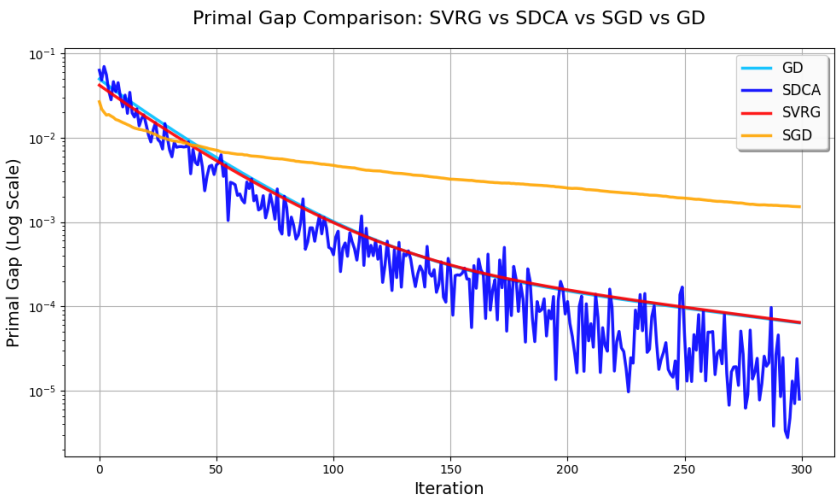

Figure 3: The primal gaps for all training losses: GD, SDCA, SVRG and SGD in log scale. Each iteration corresponds to $O(n)$ gradient computations.

Table 1 summarizes the theoretical convergence rates and computational costs for the optimization methods discussed. While Gradient Descent (GD) achieves a linear convergence rate, its $O(nd)$ per-iteration cost is often prohibitive for large-scale datasets. Stochastic Gradient Descent (SGD) reduces the per-iteration cost to $O(d)$, but the variance introduced by stochastic sampling limits its convergence to a sublinear rate of $O(1/\epsilon)$.

Method	Time per Iteration	Memory Cost	Convergence Rate
GD	$O(nd)$	$O(d)$	$O(\log(1/\epsilon))$
SGD	$O(d)$	$O(d)$	$O(1/\epsilon)$
SVRG	$O(d)^*$	$O(d)$	$O(\log(1/\epsilon))$
SDCA	$O(d)$	$O(n+d)$	$O(\log(1/\epsilon))$

Table 1: Comparison of computational and storage costs for optimization algorithms. For SVRG, $O(d)^*$ represents the amortized cost per stochastic step, which excludes the overhead of the full gradient snapshot computed at the start of each epoch.

In addition, we highlight the specific implementation details that distinguish these variance-reduction methods:

- **SDCA:** The standard implementation utilizes an epoch-based approach. In each epoch, the algorithm iterates through a random permutation of the dataset samples, updating the corresponding dual variables. This requires maintaining $O(n)$ dual variables in memory, representing the primary storage trade-off.

- **SVRG:** This method utilizes a nested loop structure. At the start of each outer loop (epoch), a full gradient $\nabla F(\tilde{w})$ is computed, costing $O(n)$. During the inner loop of m iterations, the algorithm performs stochastic updates. The $O(n)$ cost of the full gradient is *amortized* across these m steps. Thus, while any single inner-loop update is $O(d)$, the average cost per step across the entire epoch remains $O(d)$, allowing SVRG to achieve the linear convergence of GD with the per-step efficiency of SGD.

Remark. *Due to the fast convergence of the algorithms, we set the SVRG inner loop to $m = 20$, making the total number of gradient evaluations in one SVRG outer loop equivalent to one SDCA epoch ($n \approx n + m$). This allows us to say that one SDCA epoch is equivalent to one SVRG step for plotting as both use $O(n)$ gradient computations. Furthermore, we note that SDCA typically employs the dual gap as a stopping criterion, which provides a rigorous upper bound on the primal gap. This more stringent requirement often leads to a higher number of epochs for SDCA. However, when both algorithms are evaluated using the same stopping criterion—specifically the difference between consecutive weight updates—we observe a similar number of total steps on average for both algorithms, confirming their comparable computational efficiency.*

Appendix: Hyperparameter Search

SGD Hyperparameter Search

We note also that the α dampening parameter was chosen using a grid search. Otherwise, using the theoretical SGD where α is equal to 1 gives suboptimal results due to the rapid decrease of $1/t$: the test loss is on average equal to 0.1012. The reported results use a constant α equal to 0.03. The learning rate is $1/(1 + \alpha * t)$.

Learning Rate η	Regularization λ	Test Loss	Accuracy	Iterations
1×10^{-4}	1×10^{-5}	0.1053	0.8794	1779.7
1×10^{-4}	1×10^{-6}	0.1067	0.8794	951.7
5×10^{-4}	1×10^{-1}	0.0879	0.8813	256.7
5×10^{-4}	1×10^{-3}	0.0856	0.8811	525.3
5×10^{-4}	1×10^{-6}	0.0862	0.8809	598.0
1×10^{-3}	1×10^{-2}	0.0778	0.8895	309.0
1×10^{-3}	1×10^{-4}	0.0793	0.8876	193.0
1×10^{-3}	1×10^{-6}	0.0782	0.8893	301.7
1×10^{-2}	1×10^{-1}	**0.0756**	**0.8965**	246.0

Table 2: Representative results from the hyperparameter search for SGD on the Bank Marketing dataset. Each configuration was run 3 times, with mean values reported. Best test loss and best accuracy are highlighted in bold. Iteration every 50 steps.

SVRG Hyperparameter Search

Learning Rate η	Regularization λ	Test Loss	Accuracy	Iterations
1×10^{-3}	1×10^{-6}	**0.0737**	0.8956	128.7
1×10^{-3}	1×10^{-4}	0.0738	**0.8958**	128.7
1×10^{-3}	1×10^{-5}	0.0738	0.8958	132.0
1×10^{-3}	1×10^{-3}	0.0739	0.8955	132.7
1×10^{-3}	1×10^{-2}	0.0740	0.8954	132.3
5×10^{-4}	1×10^{-6}	0.0742	0.8955	217.3
5×10^{-4}	1×10^{-5}	0.0742	0.8952	218.0
5×10^{-5}	1×10^{-6}	0.0791	0.8883	1026.7
5×10^{-5}	1×10^{-1}	0.0805	0.8867	933.0
1×10^{-1}	1×10^{-2}	0.0767	0.8919	669.0
1×10^{-1}	1×10^{-1}	0.0780	0.8900	667.3

Table 3: Representative results from the hyperparameter search for SVRG on the Bank Marketing dataset. Each configuration was run 3 times, with mean values reported. The top 7 configurations by test loss are shown, along with 4 additional configurations for comparison. Best test loss and best accuracy are highlighted in bold. Iteration every 20 steps (20 is the inner-loop length of SVRG).

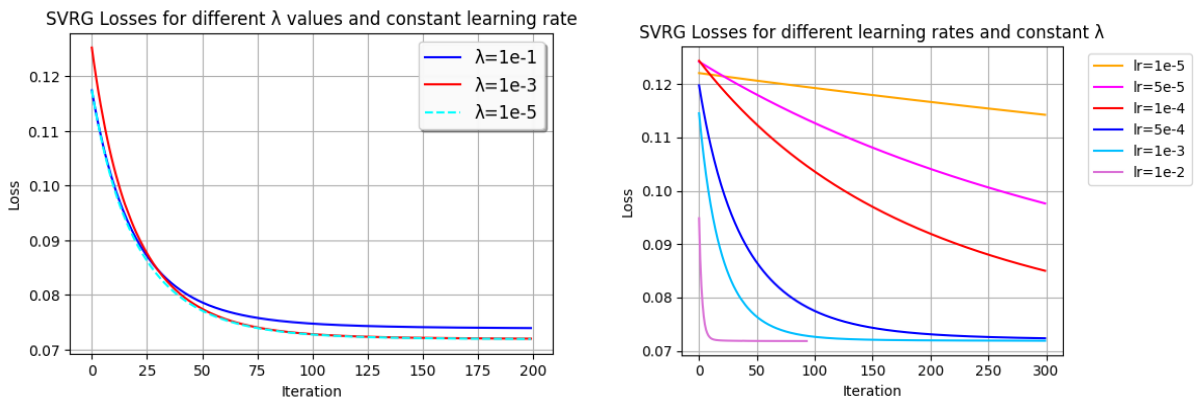

Figure 4: a) The left image shows the convergence of the losses for different lambda values but constant learning rate (1e-3) for SVRG b) The right image shows the curves of the loss when varying the learning rate and keeping lambda constant (1e-3) for SVRG

SDCA Hyperparameter Search

Modified-SGD	Regularization λ	Test Loss	Accuracy	Iterations
Yes	1×10^{-1}	0.0747	0.8965	1
Yes	1×10^{-2}	0.0734	0.8966	5
Yes	1×10^{-3}	0.0732	0.8972	38
Yes	1×10^{-4}	0.0732	0.8970	377
Yes	1×10^{-5}	0.0736	**0.8984**	999
No	1×10^{-1}	0.0746	0.8958	2
No	1×10^{-2}	0.0734	0.8966	5
No	1×10^{-3}	0.0732	0.8968	38
No	1×10^{-4}	**0.0732**	0.8970	396
No	1×10^{-5}	0.0738	0.8964	1000

Table 4: Representative results from the hyperparameter search for SDCA on the Bank Marketing dataset. The Modified-SGD option initializes the dual variables using a mini-batch SGD warm-start. Best test loss and best accuracy are highlighted in bold. Iteration every epoch (n steps), but with cheap updates (no full gradient computed).

GD Hyperparameter Search

Learning Rate η	Regularization λ	Test Loss	Accuracy	Iterations
1.0×10^{-5}	1.0×10^{-3}	0.1038	0.8794	10000
5.0×10^{-5}	1.0×10^{-2}	0.0789	0.8886	9990
1.0×10^{-4}	1.0×10^{-5}	0.0764	0.8921	6650
5.0×10^{-4}	1.0×10^{-6}	0.0741	0.8951	2170
1.0×10^{-3}	1.0×10^{-4}	0.0739	0.8955	1270
5.0×10^{-3}	1.0×10^{-3}	0.0734	0.8968	330
1.0×10^{-2}	1.0×10^{-3}	0.0734	0.8969	230
1.0×10^{-1}	1.0×10^{-3}	**0.0732**	**0.8970**	60

Table 5: Representative results from the GD hyperparameter search on the Bank Marketing dataset. Best accuracy and test loss are highlighted in bold. Iteration every step, but the steps are highly expensive in terms of gradient computations.

References

Bottou, L., F. E. Curtis, and J. Nocedal (2018). "Optimization Methods for Large-Scale Machine Learning". **SIAM Review**.

Johnson, R. and T. Zhang (2013). "Accelerating Stochastic Gradient Descent Using Predictive Variance Reduction". **Advances in Neural Information Processing Systems**.

Nesterov, Y. (2013). **Introductory Lectures on Convex Optimization**. Springer.

Robbins, H. and S. Monro (1951). "A Stochastic Approximation Method". **Annals of Mathematical Statistics**.

Shalev-Shwartz, S. and T. Zhang (2013). "Stochastic Dual Coordinate Ascent Methods for Regularized Loss Minimization". **Journal of Machine Learning Research**.

UCI Machine Learning Repository (2012). **Bank Marketing Data Set**. `https://archive.ics.uci.edu/dataset/222/bank+marketing`. Accessed: 2025-01.